

BASE DE DATOS I

Unidad 1: **Introducción a las Bases de Datos**

Definición de un sistema de base de datos



Definición	Un sistema de base de datos , también conocido como DBMS (DataBase Management System), es un conjunto de datos estructurados e interrelacionados junto con el software que permite acceder a dichos datos.
------------	---

El objetivo principal de un sistema de base de datos es permitir almacenar y recuperar los datos así como también la manipulación de los mismos para producir información.

Podemos decir que una base de datos es un objeto estructurado que consta de 2 partes, por un lado una colección de datos relacionados y organizados en tablas (finalmente terminan siendo archivos físicos dentro una pc) y por otro una serie de metadatos que describen la estructura de los datos dentro de la base de datos. Por ejemplo, si quisiéramos almacenar datos de alumnos de un establecimiento educativo, el nombre, la edad y curso al que asiste serían los datos, mientras que el nombre que le asigne a cada atributo, el tipo de dato, longitud, etc. serían los metadatos.

Definición	Un modelo de base de datos es más un concepto que un objeto físico. Especifica cómo van a estar organizados y cómo se van a relacionar los datos dentro de la base de datos.
------------	---

Definición: Componentes de un sistema de base de datos

Un sistema de base de datos está formado principalmente por 2 grandes componentes, por un lado el *gestor de almacenamiento* y por otro el *procesador de consultas*.

Gestor de almacenamiento

El gestor de almacenamiento es un módulo cuya función principal es almacenar, recuperar y actualizar los datos en la base de datos. Proporciona de forma indirecta una interfaz entre los datos (bajo nivel) y los programas de aplicación.

Este componente recibe instrucciones en DML (Data Manipulation Language) y las traduce a instrucciones de un sistema de archivos.

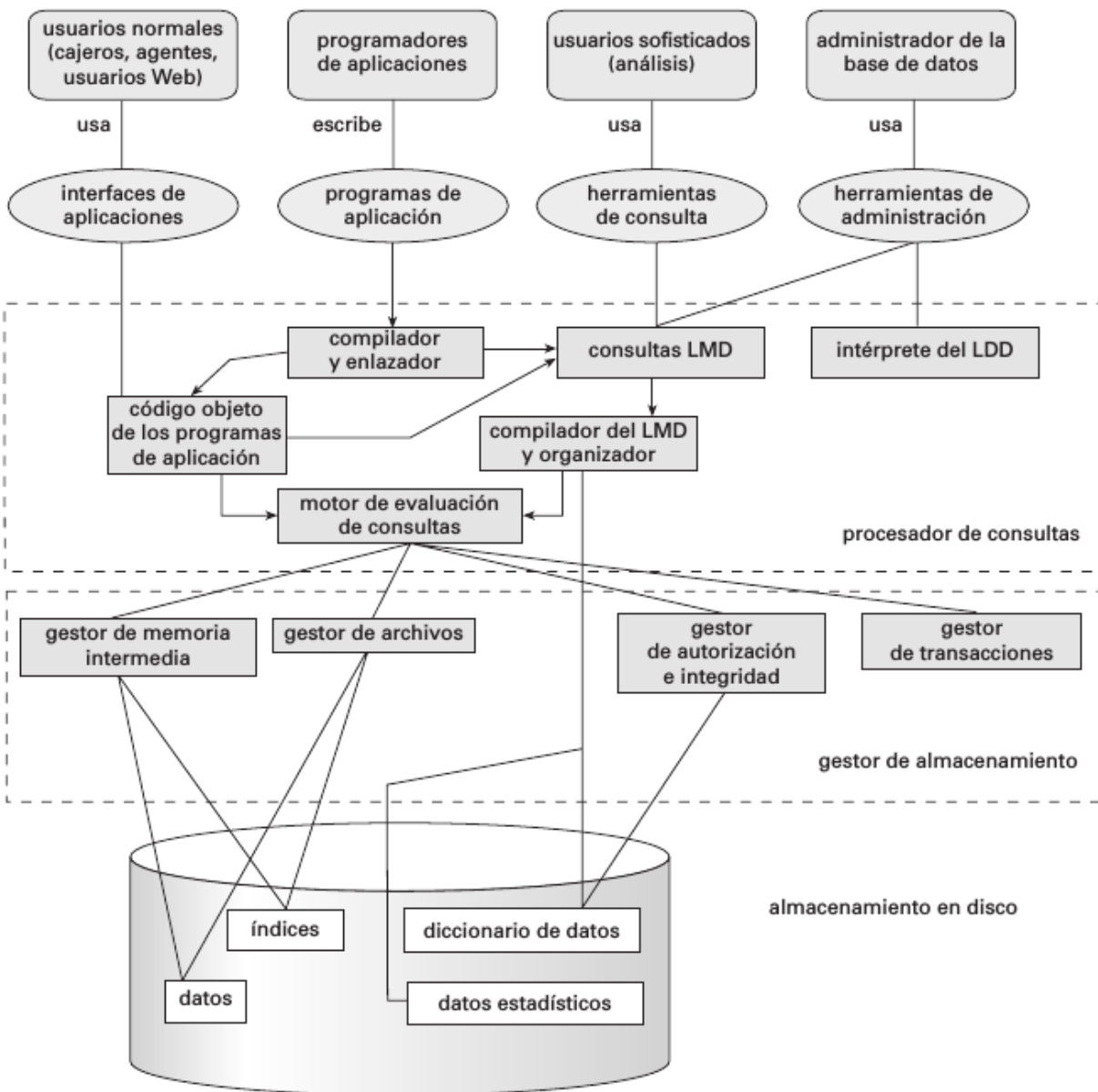
El gestor de almacenamiento, a su vez está compuesto por:

- Gestor de archivos, encargado de gestionar las operaciones contra el disco rígido
- Gestor de memoria, encargado de traer los datos desde el disco a la memoria principal (RAM). También gestiona el almacenamiento de datos en memoria caché

Procesador de consultas

Está conformado por:

- Intérprete de DDL (Data Definition Language)
- Compilador de DML, encargado de traducir instrucciones en DML a un plan de evaluación que se pasa al *motor de evaluación de consultas*. Puede generar más de 1 plan de evaluación de consultas y también puede producir algunas optimizaciones sobre las consultas.
- Motor de evaluación de consultas, encargado de ejecutar las instrucciones que recibe del *Compilador de DML*.



Ventajas del enfoque de Bases de Datos

Podemos decir que un sistema de base de datos ofrece, frente a un sistema de archivos, un control centralizado de los datos. A partir de esto se desprenden otras ventajas que se explican a continuación:

- **Compartir datos**



Una base de datos permite que varias aplicaciones puedan acceder a los datos y que se desarrollen nuevas aplicaciones que también pueden acceder a los datos sin tener que modificar la información contenida en la base de datos.

- **Reducir la redundancia**

Sin una base de datos, cada aplicación posee sus propios archivos. Esto facilita la redundancia de los datos con el consecuente desperdicio de espacio de almacenamiento. No toda redundancia de datos es mala, hay casos en los que es necesaria pero debe estar controlada.

- **Evitar inconsistencias**

Las inconsistencias entre los datos se dan principalmente cuando el mismo dato se encuentra representado en más de 1 entidad dentro de la base de datos. De esta manera en una entidad podría tener unos datos y en la otra entidad otros distintos. Las inconsistencias quedan prácticamente eliminadas si se aplica un control efectivo de la redundancia (o se elimina).

- **Manejo de transacciones**

Una transacción es una unidad de trabajo lógica, que por lo general comprende varias operaciones, sobre la base de datos, permitiendo aplicar todas o ninguna de ellas.

- **Mantener la integridad de los datos**

La integridad de los datos trata principalmente de la correctitud de los mismos dentro de la base de datos. La falta de integridad puede deberse a problemas de inconsistencia (producto de la redundancia), o bien por falta de controles de integridad, también conocidas como reglas de negocio.

- **Controlar la seguridad**

Una base de datos permite asegurar que el único medio de acceso sea a través de los canales adecuados y definir restricciones de acceso que se verificarán en cada operación

Independencia de los Datos

La independencia de los datos es una ventaja o mejor dicho un objetivo buscado por los sistemas de bases de datos.

Definición	Podemos definir la independencia de los datos como la inmunidad de las aplicaciones a cambios en la representación física y técnicas de acceso de los datos.
-------------------	--

Esto significa que dentro de la base de datos podemos cambiar la forma como se almacenan físicamente los datos, el tipo de indexado de los mismos, inclusive el dato indexado, etc. sin tener que modificar el código de la aplicación que accede a los mismos.

Ejemplo:

Imagine una aplicación que administre datos de alumnos de un establecimiento educativo. Si por motivos de rendimiento, se necesitará implementar un indexado de los datos sobre el campo *nombre*, la aplicación debería ser modificada para tal fin.

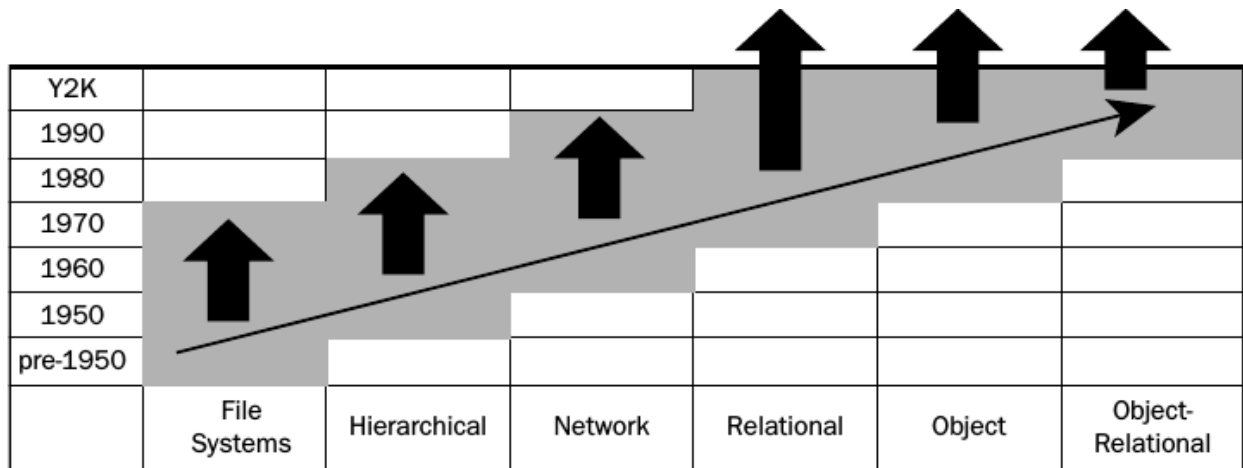
En un sistema de bases de datos, este cambio sería transparente para la aplicación, ya que el mismo se implementaría directamente en la base de datos sin modificar la aplicación.

Otros casos de independencia de datos podrían ser:

- **Representación de los datos**
 - Campos numéricos: podría ser necesario cambiar la base (binaria, decimal, etc); almacenar por un lado la parte decimal y por otro la parte entera, etc.
 - Campos alfanuméricos: cambiar la codificación de los caracteres (Unicode, UTF-8, ISO-8859-1, etc)
- **Codificación de los datos**
 - En algunos casos es deseable codificar los datos, por ejemplo un campo que guarda una cadena de caracteres Rojo, Verde, Azul podría codificarse como 1, 2, 3 respectivamente.
- **Materialización de los datos**
 - En ocasiones el dato que “ve” la aplicación no es necesariamente el que está almacenado en la base de datos. Ejemplos de este tipo surgen cuando el campo solicitado por la aplicación es producto de un cálculo que realiza la base de datos.

La independencia de los datos también implica que la base de datos pueda crecer sin tener que modificar las aplicaciones que acceden a la misma. Imagine que sobre una entidad *alumno* se agregan atributos como ser *telefono_celular*, *email*. Si estos atributos no son requeridos por una aplicación, la misma no debería verse afectada.

Historia y evolución de los sistemas de bases de datos.



Sistema de archivos (File Systems)

En este tipo de sistema, los datos son almacenados en archivos planos (texto o binarios) utilizando la estructura del sistema de archivos proporcionada por el sistema operativo.

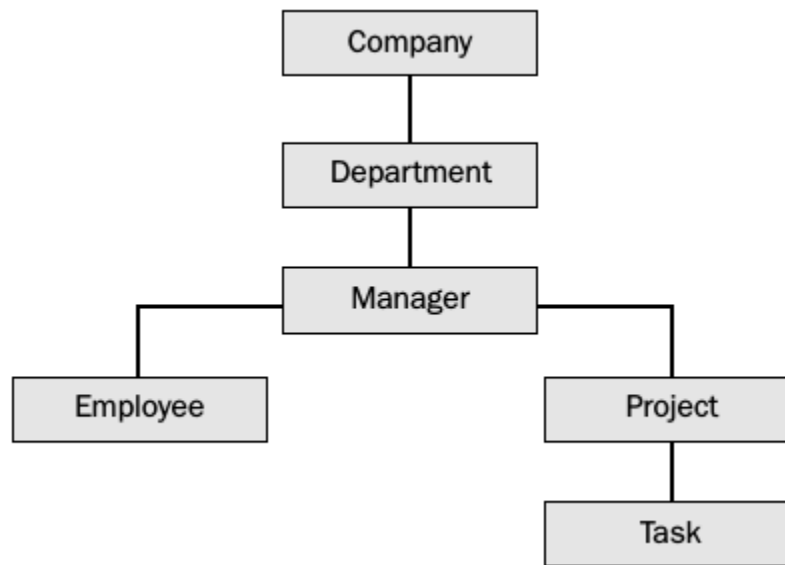
Cualquier acceso a los datos (lectura, escritura y/o, validaciones debe ser explícitamente codificada. Los datos pueden ser almacenados en varios archivos y al igual que las búsquedas cualquier relación entre los datos y/o validaciones debe ser programada.

La capacidad de crecimiento de estos sistemas es limitada.

Jerárquico (Hierarchical)

Este modelo de base de datos, presenta una estructura de árbol invertida. Las tablas de este modelo establecen una relación padre-hijo donde cada tabla hijo puede tener un único padre y cada tabla padre puede tener múltiples hijos.

La mejora presentada respecto del modelo de sistema de archivos es que se establecen relaciones uno a muchos (1 - N)



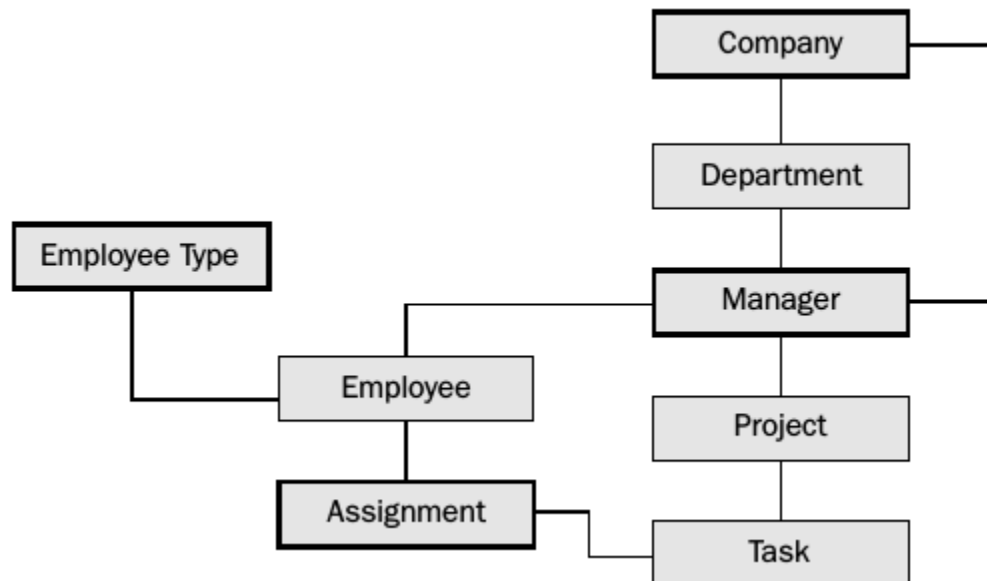
En el ejemplo cada tarea es parte de un proyecto, que a su vez pertenece a un administrador que es parte de un departamento.

Algunas desventajas son que para acceder a un dato hay que conocer toda la estructura, y otra es la rigidez del modelo ya que el hecho de tener que agregar una tabla puede ser un trabajo tedioso si no se tuvo en cuenta al momento del diseño inicial.

Red (Network)

Este modelo es básicamente un refinamiento sobre el modelo jerárquico. En este modelo se permite que una tabla hijo tenga más de una tabla padre.

La mejora presentada respecto del modelo de sistema jerárquico es que se permite establecer relaciones muchos a muchos (N - M)



En el ejemplo se puede ver como los *administradores* pueden ser parte tanto de una *compañía* como de un *departamento*.

Vemos que se agrega la tabla *tipos de empleados* y *asignaciones* para poder definir las asignaciones de cada empleado.

Más allá de tener la posibilidad de definir relaciones N-M, necesitamos definir una única combinación entre padres e hijos, eso se resuelve agregando una tabla intermedia. En el ejemplo la tabla *asignaciones* se agrega para resolver las múltiples combinaciones entre *empleados* y *tareas*. Sin esta tabla intermedia sería imposible encontrar una única asignación de tareas.

Si bien este modelo es más flexible que el jerárquico posee una de las desventajas del mismo que es tener que conocer la estructura completa del modelo para poder acceder a los datos.

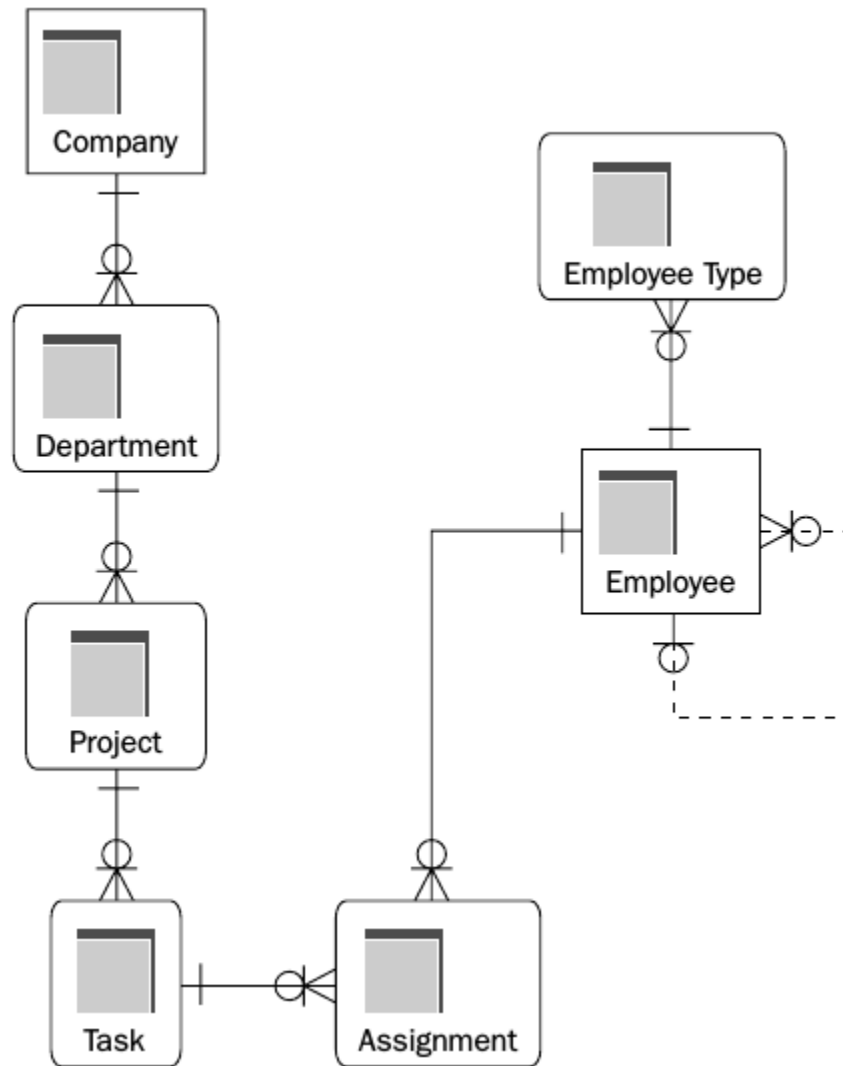
Relacional (Relational)

El modelo relacional, surge para mejorar las restricciones de los modelos anteriores, sin abandonar completamente la estructura jerárquica.

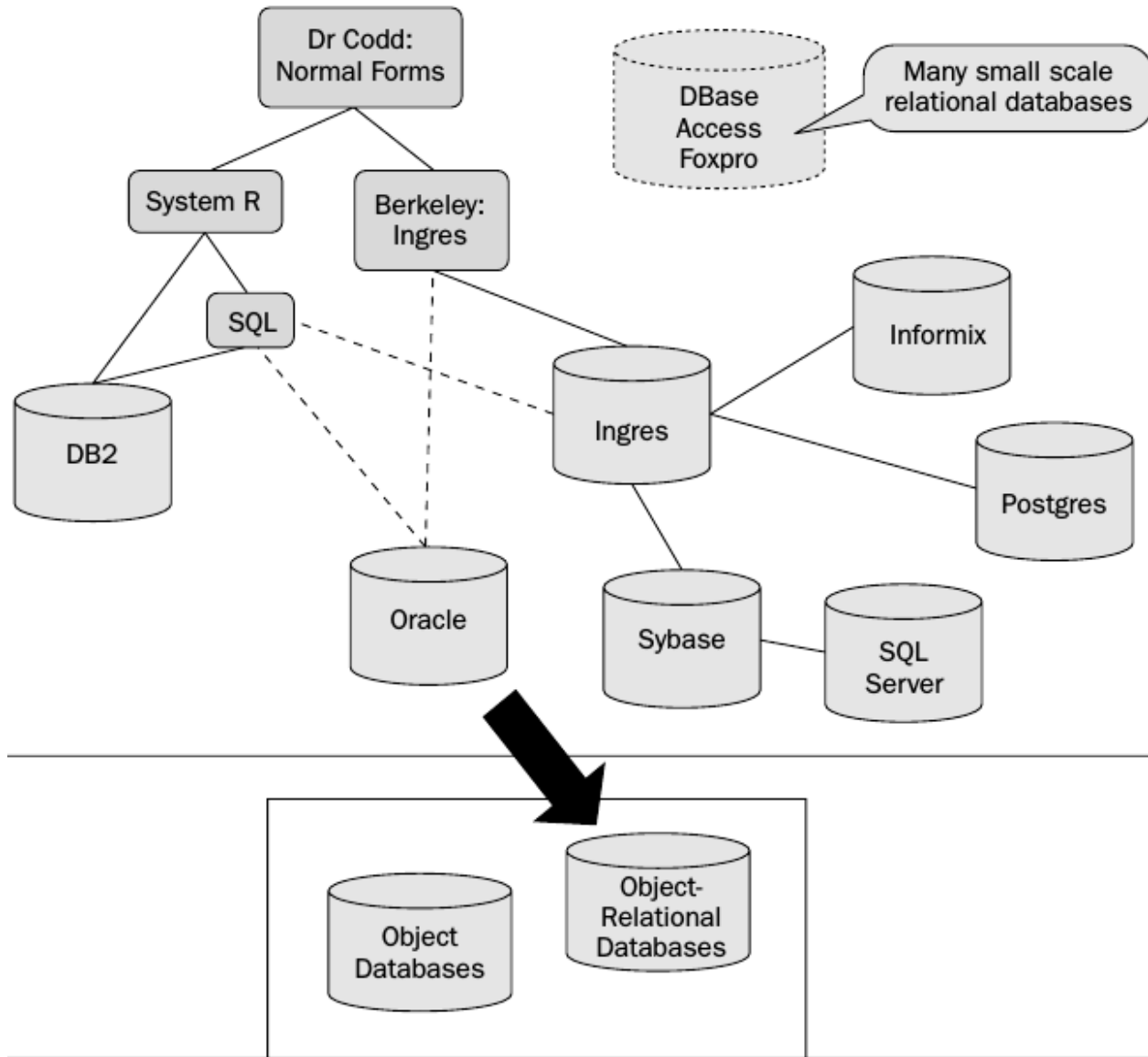
Cualquier dato puede ser accedido sin necesidad de recorrer la estructura completa de tablas, sólo basta con saber qué se está buscando.

Otro de los beneficios es que las relaciones entre tablas no tienen en cuenta su posición jerárquica, cualquier tabla puede ser relacionada con otra siempre y cuando se cuente con los datos apropiados.

Este modelo es más flexible que los anteriores permitiendo agregar tablas y relaciones en cualquier momento sin la complejidad de reestructurar todo el modelo.



En el ejemplo vemos el modelo de datos de proyecto y tareas llevado a un modelo relacional. Se puede ver la relación 1 - N entre Proyecto y Tareas y como la relación entre Empleados y Tareas (N - M) se resuelve con la tabla de asignaciones realizando 2 relaciones 1 - N.



Historia de las bases de datos relacionales

Objeto (Object)

El modelo de datos orientado a objetos es una adaptación para los sistemas de bases de datos del paradigma de la programación orientada a objetos. Estos sistemas son eficientes para acceder a un único dato, pero ineficientes a la hora de devolver varios datos.



Estos modelos resuelven la complejidad de una relación N - M, de las bases de datos relacionales, utilizando colecciones

Objeto-Relacional (Object-Relational)

El modelo objeto-relacional trata de unificar los beneficios de ambos modelos. Básicamente lo que se trata de hacer es incluir las capacidades de un modelo orientado a objetos dentro de un sistema de bases de datos relacional.

NoSQL

Los modelos de bases de datos NoSQL, también llamado “no relacional”, existen desde la década de los 60, pero comenzaron a ser masivamente utilizadas con el advenimiento de la web 2.0. Algunas características que motivó el uso de estos modelos es la flexibilidad en la definición de las estructuras de datos, la velocidad de acceso.

Los motivos para su tardía adopción se deben a que muchos de estos modelos carecen de mecanismos de control de integridad, control de transacciones en beneficio de la velocidad de acceso de los datos.

Tipos de base de datos

Cuando diseñamos y desarrollamos una aplicación, el objetivo principal es proveer una interfaz amigable para el usuario final. Los usuarios definen requerimientos para la aplicación que necesitan y muchos de estos definirán restricciones que derivarán en el tipo de base de datos a utilizar.

A nivel funcional los tipos de base de datos pueden ser:

- Transaccionales
- Sistemas para la toma de decisión (DSS)
- Híbridos

Transaccionales

También conocidas como OLTP (online transaction processing), son bases de datos que basan su operatoria en pequeños cambios a la base de datos. Lo que se trata de hacer es que las operaciones sean atómicas (transacciones) para poder mantener la consistencia de los datos. La función principal de la base de datos es agregar, modificar o eliminar pequeñas cantidades de datos, es decir registros.

Sistemas para la toma de decisión

También conocidos como OLAP (online analytical processing), son sistemas de bases de datos que dan soporte a los altos mandos de una organización para tomar una mejor decisión fundamentada en la información de sus sistemas. En esta categoría se encuentran los sistemas: *data warehouse* que toman como entrada millones de datos históricos de un sistema OLTP (datos que se copian desde el sistema OLTP) para producir información estadística sobre algún aspecto.

Híbridos

Un sistema híbrido, es una mezcla de un sistema OLTP con un sistema *data warehouse*. En casos donde el volumen de datos no es muy grande puede ser una buena opción costo-beneficio.

OLTP vs OLAP

	OLTP	OLAP
Fuente de datos	Datos de operaciones. Proviene de las aplicaciones de usuario	Datos consolidados. Proviene de varios sistemas OLTP
Propósito de los datos	Tareas operativas y de control	Planificación, toma de decisión
Qué significan/muestran los datos	Una foto del estado del negocio	Son múltiples vistas que sobre las distintas actividades del negocio
Actualización de datos	Operaciones cortas y rápidas	Operaciones periódicas y por lote
Consultas	Relativamente simples y rápidas devolviendo pocos registros	A menudo son consultas complejas que incluyen operaciones de consolidación de datos
Velocidad de procesamiento	Operaciones muy rápidas	Depende de la cantidad de datos involucrados, por lo general los procesos en lote demoran horas
Almacenamiento	Con un buen manejo de datos históricos podría no requerir mucho espacio	Requiere mucho espacio de almacenamiento debido a la existencia de datos históricos, índices, etc.
Diseño de la base de datos	Datos Normalizados	Por lo general los datos se encuentran desnormalizados



Backup y recupero	Deben realizarse con mucha frecuencia. Una pérdida de los datos operacionales puede ocasionar pérdidas económicas, problemas legales, etc.	Políticas menos rigurosas que los sistemas OLTP, ya que los datos pueden volver a cargarse.
--------------------------	--	---

Conclusión

Las aplicaciones suelen tener una mezcla de necesidades de procesamiento de transacciones y ayuda a la toma de decisiones. Por tanto, el mejor sistema de bases de datos para cada aplicación depende de la mezcla de las dos necesidades que tenga la aplicación.





Bibliografía

- Gavin Powell. Beginning Database Design. Indianapolis: Wiley Publishing, Inc, 2006. ISBN-13: 978-0-7645-7490-0.
- Abraham Silberschatz; Henry F. Korth; S. Sudarshan. FUNDAMENTOS DE BASES DE DATOS. Cuarta Edición. España: McGRAW-HILL/INTERAMERICANA DE ESPAÑA, S. A. U, 2002.
- C. J. Date. Introducción A Los Sistemas De Bases De Datos. Séptima Edición. Méjico: Pearson Educación, 2001. ISBN: 968-444-419-2



Bibliografía:

- Diseño y Administración de Base de Datos Ing. Alejandro Mostovoi CUVL- 2018
- GavinPowell. Beginning Database Design. Indianapolis: Wiley Publishing, Inc, 2006. ISBN-13: 978-0-7645-7490-0.
- Abraham Silberschatz; Henry F. Korth; S. Sudarshan. FUNDAMENTOS DE BASES DE DATOS. Cuarta Edición. España: McGRAW-HILL/INTERAMERICANA DE ESPAÑA, S. A. U, 2002.
- C.J.Date. Introducción A Los Sistemas De Bases De Datos. Séptima Edición. Méjico: Pearson Educación, 2001. ISBN: 968-444-419-2